Mathématiques numériques — Série 1

Corrigé détaillé (MATLAB)

Richard Baltensperger — HEIA-FR, 2026-2027

Philippe — E2-p2, Génie électrique

18 septembre 2026

Source : `Consignes/sources/matnum-serie1-exercices.pdf` (10 exercices) et `matnum-serie1-solutions.pdf` (solutions du professeur).
*Toutes les sorties reproduites ci-dessous proviennent de l'exécution réelle du code dans **MATLAB R2025a Update 1**, jamais d'un calcul de tête.*
Scripts : `scripts/serie1_exercices.m` (les 10 exercices) et `scripts/serie1_pieges.m` (cas limites et vérifications). Les valeurs sont reproduites telles quelles ; celles que MATLAB affiche une par ligne ont parfois été regroupées pour la mise en page.

---

1. Format `long` : toutes les décimales de π et de e

Énoncé. Utiliser `format long` pour trouver toutes les décimales de π et e utilisées en interne dans MATLAB.

1.1 Méthode

`format long` change uniquement *l'affichage* : les nombres restent des **doubles IEEE 754** (1 bit de signe, 11 bits d'exposant, 52 bits de mantisse, soit 53 bits de précision). Il affiche 15 chiffres significatifs ; pour en voir davantage, imposer le format d'impression.

1.2 Code et résultat

```
format long
pi
exp(1)
```

```
ans = 3.141592653589793
ans = 2.718281828459045
```

Pour forcer l'affichage de 17 chiffres significatifs (script `serie1_pieges.m`) :

```
fprintf('%.17g\n', pi)
fprintf('chiffres utiles ~ %.2f\n', -log10(eps))
```

```
3.1415926535897931
chiffres utiles ~ 15.65
```

1.3 Que stocke réellement la machine

La valeur *exacte* des doubles les plus proches de π et de e (conversion décimale exacte, vérifiée) est :

$$\pi_{\text{double}} = 3{,}14159265358979311599796346854418516159057617 5 \ldots$$

$$e_{\text{double}} = 2{,}71828182845904509079559829842764884233474731445 3125$$

On a bien $\pi - \pi_{\text{double}} = 0$ en double : l'affichage `format long` s'arrête avant les décimales qui trahissent le binaire (… 5897931 1599 …).

À retenir.

- `format long` affiche la valeur arrondie à **15 chiffres significatifs** : ce n'est pas le contenu exact de la mémoire.
- L'écart de représentation est mesuré par `eps` $= 2^{-52} = 2{,}220446049250313 \times 10^{-16}$: environ **15,65 chiffres décimaux** utiles (17 pour garantir un aller-retour exact).
- Pour imprimer 17 chiffres : `fprintf('%.17g', x)` ou `num2str(x, 17)`.
- `format short` (défaut) revient à 5 chiffres significatifs.

2. Format `rat` : approximation rationnelle

Énoncé. Utiliser `format rat` pour trouver une approximation rationnelle de π et e.

2.1 Méthode

`rat(x)` décompose x en **fraction continue** et renvoie le couple `[n, d]` de numérateur/dénominateur : c'est l'équivalent calculable de l'affichage `format rat`. La recherche s'arrête dès que l'erreur absolue descend sous une **tolérance** `tol` (par défaut tol $= 10^{-6}\,\|x\|$, soit $3{,}142 \times 10^{-6}$ pour π et $2{,}718 \times 10^{-6}$ pour e).

2.2 Code et résultat

```
[n_pi, d_pi] = rat(pi)
[n_e, d_e] = rat(exp(1))
```

```
n_pi = 355    d_pi = 113
n_e  = 1457   d_e  = 536
erreurs absolues : 2.668e-07 (pi) et 1.754e-06 (e), soit 8.49e-08 et 6.45e-07 en relatif
```

2.3 Le rôle de la tolérance (démonstration)

```
for tol = [1e-4 1e-6 1e-8 1e-10 1e-12]
    [n, d] = rat(exp(1), tol);
    fprintf('rat(e, %g) = %d/%d  (erreur %.3e)\n', tol, n, d, abs(n/d - exp(1)))
end
```

```
rat(e, 0.0001) = 193/71  (erreur 2.803e-05)
rat(e, 1e-06) = 2721/1001  (erreur 1.102e-07)
rat(e, 1e-08) = 25946/9545  (erreur 5.515e-09)
rat(e, 1e-10) = 566827/208524  (erreur 1.154e-11)
rat(e, 1e-12) = 1084483/398959  (erreur 4.818e-13)
```

La fraction renvoyée par défaut (1457/536) est donc *moins* précise que 2721/1001 : elle satisfait simplement la tolérance relative de 10^{-6}, atteinte plus tôt.

À retenir.

- 355/113 est la **meilleure approximation rationnelle** de π à dénominateur < 1000 (erreur relative $8{,}5 \times 10^{-8}$) ; 1457/536 est celle de e fournie par la tolérance par défaut.
- `format rat` modifie l'*affichage* ; `[n,d] = rat(x, tol)` renvoie les *entiers* : c'est cette forme qu'il faut utiliser dans un calcul.
- `pi` est une fraction continue remarquable : $[3; 7, 15, 1, 292, 1, 1, 1, 2, \ldots]$; ses réduites successives sont 22/7, 333/106, 355/113, 103993/33102.

Piège. $\dfrac{1457}{536}$ n'est pas une valeur « remarquable » de e et le résultat **dépend de la tolérance** : énoncer une fraction sans préciser la tolérance n'a pas de sens. Pour un test, imposer explicitement `rat(x, 1e-10)`.

3. Qu'est-ce qui est plus grand : e^{π} ou π^e ?

Énoncé. Comparer e^{π} et π^e.

3.1 Méthode : comparer les logarithmes

ln est strictement croissante sur $]0, +\infty[$, donc comparer a et b revenient à comparer $\ln a$ et $\ln b$. En utilisant $\ln(u^v) = v \ln u$:

$$\ln(e^{\pi}) = \pi, \qquad \ln(\pi^e) = e \ln(\pi).$$

```
exp(pi)
pi^exp(1)
exp(pi) > pi^exp(1)
```

```
ans = 23.140692632779267
ans = 22.459157718361041
ans = logical 1
```

L'écart et le contrôle par le logarithme (`serie1_pieges.m`) :

```
exp(pi) - pi^exp(1)
pi - exp(1)*log(pi)
```

```
e^pi - pi^e = 0.681534914418226
pi - e*log(pi) = 0.029894206391372
```

3.2 Preuve générale

Posons $f(x) = \dfrac{\ln x}{x}$ pour $x > 0$:

$$f'(x) = \frac{1 - \ln x}{x^2}, \qquad f'(x) < 0 \iff x > e.$$

f est donc **strictement décroissante sur** $]e, +\infty[$. Comme $\pi > e$:

$$\frac{\ln \pi}{\pi} < \frac{\ln e}{e} = \frac{1}{e} \implies e \ln \pi < \pi \implies \boxed{\pi^e < e^{\pi}}$$

Vérification numérique des deux membres : $\dfrac{\ln \pi}{\pi} = 0{,}3643788397$ contre $\dfrac{1}{e} = 0{,}3678794412$. Le maximum de f sur $[2, 4]$ vaut $0{,}367879441171359$, atteint en $x = 2{,}71828 \approx e$: e est bien l'abscisse du maximum.

À retenir. $e^{\pi} > \pi^e$ avec une marge confortable : $e^{\pi} - \pi^e = 0{,}681534914418223532\ldots$ (30 chiffres significatifs). Aucun risque d'erreur d'arrondi en double, la comparaison `exp(pi) > pi^exp(1)` est fiable.

Piège. Le raisonnement « $e < \pi$ donc $e^{\pi} < \pi^e$ » est **faux** : les deux exposants diffèrent. Toute comparaison de puissances à bases différentes passe par les logarithmes.

4. Vecteur ligne, transposée et `x(:)`

Énoncé. Définir un vecteur ligne x contenant $1, \ldots, 10$, utiliser la transposée `'` pour le rendre colonne, puis expliquer `x = x(:)`.

4.1 Code et résultat

```
x = 1:10;
size(x)
```

```
x = x';
size(x)
y = 1:5;
size(y(:))
```

```
ans = 1 10      % vecteur ligne
ans = 10 1      % vecteur colonne
ans = 5 1       % x(:) rend toujours une colonne
```

Aplatissement d'une matrice et les deux opérateurs de transposition (`serie1_pieges.m`) :

```
A = [1 2 3; 4 5 6];
A(:)'
v = 1 + 2i;
v'
v.'
w = 1:3; wc = w';
isequal(w, wc)
isequal(w(:), wc(:))
```

```
ans = 1 4 2 5 3 6                % A(:) lit colonne par colonne
ans = 1.000000000000000 - 2.000000000000000i   % v'  : conjuguee
ans = 1.000000000000000 + 2.000000000000000i   % v.' : non conjuguee
isequal(w, wc) = 0 | isequal(w(:), wc(:)) = 1
```

4.2 Interprétation

`x(:)` **linéarise** le tableau en un vecteur **colonne**, en parcourant la mémoire **colonne par colonne** (ordre *column-major* de MATLAB). Il est idempotent : appliqué à un vecteur colonne, il ne change rien. C'est le réflexe à avoir dès qu'une fonction exige un vecteur colonne (ou pour agréger un tableau : `sum(A(:))`, `max(A(:))`).

À retenir.

- `'` est la transposée **conjuguée** (`ctranspose`) ; `.'` la transposée simple. Sur des réels, identiques ; sur du complexe, `(1+2i)' = 1-2i`.
- `size(x)` renvoie `[lignes colonnes]` : `size(x,1)` = nombre de lignes.
- Deux vecteurs de mêmes valeurs mais d'orientations différentes ne sont **pas égaux** pour `isequal` (`0` ci-dessus) : comparer via `isequal(x(:), y(:))`.

5. Produit scalaire de deux vecteurs

Énoncé. Calculer $\sum_{k=1}^{100} x_k y_k$ avec $x = (1, 2, \dots, 100)$ et $y = (1, 2, 1, \dots, 1)$ (tous deux de taille 1×100).

5.1 Méthode

Le produit scalaire est le produit matriciel d'un vecteur ligne par un vecteur **colonne** (taille $(1\times100)\cdot(100\times1) = 1\times1$, donc un scalaire) :

$$x\,y^{\mathsf{T}} = \sum_{k=1}^{100} x_k y_k.$$

```
x = 1:100;
y = ones(1, 100);
y(2) = 2;
x*y'
```

```
ans = 5052
```

Contrôles et pièges de dimensions (`serie1_pieges.m`) :

```
x*y' = 5052 ; y*y' = 103 = sum(y.^2) ; taille de x'*y : 100 x 100
controles : sum(x.*y) = 5052, dot(x,y) = 5052, sum(1:100)+2 = 5052
x*y -> Incorrect dimensions for matrix multiplication. Check that the number of columns in
the first matrix matches the number of rows in the second matrix. To operate on each element
of the matrix individually, use TIMES (.*) for elementwise multiplication.
```

5.2 Détail du calcul

Sans le `y(2) = 2`, on aurait $\sum_{k=1}^{100} k = \frac{100\cdot 101}{2} = 5050$. Le deuxième terme passe de $2\cdot 1 = 2$ à $2\cdot 2 = 4$, soit $+2$:

$$5050 + 2 = \boxed{5052}$$

À retenir. $y^{\mathsf{T}}y = \sum_k y_k^2$ est le *carré* de la norme de y : ici $99\cdot 1 + 4 = 103$ (à ne pas confondre avec $\left(\sum_k y_k\right)^2 = 101^2 = 10201$).

Piège. `x*y` (sans transposée) est une multiplication matricielle impossible : *« Incorrect dimensions for matrix multiplication »*. Le produit *terme à terme* s'écrit `x.*y`. Enfin `x'*y` n'est pas un scalaire mais un **produit extérieur** 100×100 : la transposée doit être du bon côté.

6. Les factorielles 1! à 15! par `cumprod`

Énoncé. Calculer la liste des factorielles $(1!, 2!, \ldots, 15!)$ avec `cumprod`.

6.1 Méthode

`cumprod(v)` calcule les produits cumulés : le k-ième élément vaut $v_1 v_2 \cdots v_k$. Avec $v = (1, 2, \ldots, 15)$ on obtient exactement $1!, 2!, \ldots, 15!$.

```
x = 1:15;
y = cumprod(x);
fprintf('%d! = %d\n', [x; y])
```

```
1! = 1                8!  = 40320             15! = 1307674368000
2! = 2                9!  = 362880
3! = 6                10! = 3628800
4! = 24               11! = 39916800
5! = 120              12! = 479001600
6! = 720              13! = 6227020800
7! = 5040             14! = 87178291200
```

6.2 Jusqu'où les entiers restent exacts ?

$2^{53} = 9\,007\,199\,254\,740\,992$. Une factorielle est représentable *exactement* en double tant que sa partie impaire tient sur 53 bits, c'est-à-dire jusqu'à 22! :

```
18! MATLAB = 6402373705728000 | exact = 6402373705728000 | identique : 1
19! MATLAB = 121645100408832000 | exact = 121645100408832000 | identique : 1
20! MATLAB = 2432902008176640000 | exact = 2432902008176640000 | identique : 1
21! MATLAB = 51090942171709440000 | exact = 51090942171709440000 | identique : 1
22! MATLAB = 1124000727777607680000 | exact = 1124000727777607680000 | identique : 1
23! MATLAB = 25852016738884978212864 | exact = 25852016738884976640000 | identique : 0
prod(1:170) = 7.257416e+306 (fini) | prod(1:171) = Inf (debordement)
```

À retenir.

— `cumprod(x)` = produits cumulés, `prod(x)` = produit total, `factorial(n)` = $n!$ pour un scalaire. Les trois coïncident ici (`cumprod(1:15) == arrayfun(@factorial, 1:15)` vaut `1`).

— **Exactitude :** jusqu'à 22! la valeur imprimée est l'entier exact ; à partir de 23! MATLAB arrondit (… 78212864 au lieu de … 76640000). La leçon du chapitre 1 : un résultat « entier » affiché sans point décimal n'est pas pour autant exact.

— **Débordement :** $171! >$ `realmax` $\approx 1{,}8 \times 10^{308}$, donc `Inf` dès 171! ; $170! \approx 7{,}26 \times 10^{306}$ passe encore.

7. Carrés magiques `magic(n)`

Énoncé. Pour $n = 10, 20, 30$, vérifier que la somme des lignes et des colonnes de `magic(n)` est constante et vaut $(n^3 + n)/2$.

7.1 Méthode

Un carré magique d'ordre n contient les entiers 1 à n^2 une fois chacun. La somme totale vaut donc $\sum_{k=1}^{n^2} k = \frac{n^2(n^2+1)}{2}$, répartie sur n lignes :

$$S = \frac{1}{n} \cdot \frac{n^2(n^2+1)}{2} = \frac{n(n^2+1)}{2} = \frac{n^3+n}{2}. \tag{1}$$

```
for n = [10 20 30]
    T = magic(n);
    fprintf('n = %2d : constante %6d, ecart lignes/colonnes = %g / %g\n', ...
            n, (n^3 + n)/2, max(abs(sum(T) - (n^3+n)/2)), max(abs(sum(T') - (n^3+n)/2)))
end
```

```
n = 10 : constante    505, ecart lignes/colonnes = 0 / 0
n = 20 : constante   4010, ecart lignes/colonnes = 0 / 0
n = 30 : constante  13515, ecart lignes/colonnes = 0 / 0
```

7.2 Limites de `magic`

```
magic(2) : lignes [5  5], colonnes [4  6]
magic(3) : lignes [15  15  15], colonnes [15  15  15]
```

À retenir. `sum(T)` somme les *colonnes*, `sum(T')` les *lignes*, `sum(diag(T))` la diagonale principale, `sum(diag(fliplr(T)))` l'antidiagonale. La constante (1) se retrouve par simple comptage : pas besoin de la retenir.

Piège. `magic(2)` $= \begin{pmatrix} 1 & 3 \\ 4 & 2 \end{pmatrix}$ n'est **pas** un carré magique (lignes : 5 et 5 ; colonnes : 4 et 6). `magic(n)` n'est défini que pour $n \geq 3$ et pour un entier scalaire. Ne pas conclure « ça marche pour tout n » à partir du test $n = 10$.

8. Fonction `pair` : tous les éléments pairs

Énoncé. Écrire `function t = pair(a)` qui retourne 1 si tous les éléments de a sont pairs, 0 sinon (`rem` peut servir).

8.1 La solution du professeur, décortiquée

```
function t = pair(a)
    t = prod(rem(a, 2) - 1) ~= 0;
end
```

Découpage de l'expression, sur $a = (2,4,6,8)$:

Étape	expression	$a = (2,4,6,8)$	$a = (2,4,7,8)$
reste modulo 2	`rem(a,2)`	0 0 0 0	0 0 1 0
décalage	`rem(a,2)-1`	−1 −1 −1 −1	−1 −1 0 −1
produit	`prod(...)`	−1	0
test	`= 0`	1 (vrai)	0 (faux)

L'astuce : `rem(a,2)-1` vaut 0 pour un *impair* et −1 pour un *pair*. Le produit vaut donc 0 dès qu'un seul élément est impair, et ± 1 sinon.

```
ans = logical 1        % pair([2 4 6 8])
ans = logical 0        % pair([2 4 7 8])
```

Cas limites de la solution du professeur (`serie1_pieges.m`) :

```
pair([]) = 1 | pair(2.5) = 1 | all(rem([],2)==0) = 1
```

8.2 Version explicite (recommandée à l'examen)

```
function t = pair_clair(a)
    t = all(rem(a, 2) == 0);    % lisible et sans piege de produit vide
end
```

À retenir. `all(cond)` teste si *toutes* les valeurs vérifient `cond` ; `any(cond)` si au moins une. C'est la traduction directe de « pour tout » et « il existe ».

Piège. Le contrat de l'énoncé — « un vecteur d'**entiers** » — est indispensable :

- `pair([]) = 1` : `prod([]) = 1` (élément neutre), donc « tous pairs » est vrai sur un ensemble vide ;
- `pair(2.5) = 1` : `rem(2.5,2) = 0.5` donne $0{,}5 - 1 = -0{,}5 \neq 0$, mais avec le test `all(rem(a,2)==0)` le résultat est `0`. La fonction du prof ne teste pas l'intégrité de l'entrée.

9. Fonction `interleave` : entrelacer deux vecteurs

Énoncé. Écrire `function c = interleave(a, b)` qui entrelace les éléments de a et b : pour $a = (1,3,5,7)$ et $b = (-2,-4)$, on veut $c = (1,-2,3,-4,5,7)$. Le programme doit aussi fonctionner avec des vecteurs vides.

9.1 Solution du professeur

```
function c = interleave(a, b)
    n = length(a); m = length(b);
    k = min([n m]);
    c = [];
    for i = 1:k
        c = [c a(i) b(i)];
    end
```

```
    if n == k
        c = [c b(k+1:m)];
    elseif m == k
        c = [c a(k+1:n)];
    end
end
```

L'idée est juste : entrelacer jusqu'à `k = min(n,m)` éléments, puis recopier la queue du vecteur le plus long (`if n == k` signifie « *b* est le plus long »).

9.2 Ce que révèle l'exécution

En lançant ce code tel qu'il est imprimé, avec l'exemple de l'énoncé (vecteurs *colonnes*) :

```
interleave du prof -> Dimensions of arrays being concatenated are not consistent.
ecart 1 -> [1 -2  3 -4  5 -6  7] (le defaut ne se voit pas)
version verticale -> [1 -2  3 -4  5  7]
```

Cause. `c` démarre à `[]` et grossit *horizontalement* (`[c a(i) b(i)]` produit un vecteur *ligne*). Or *a*, *b* et donc la queue `a(k+1:n)` sont des vecteurs *colonnes* : concaténer une ligne (1 × 6) et une colonne (2 × 1) est impossible. Le défaut ne se manifeste donc que lorsque la **queue comporte au moins 2 éléments** (écart de longueur ≥ 2) : avec un écart de 1 la queue est un scalaire, et le test passe. Il ne s'agit pas d'une erreur d'algorithme mais d'une **coquille de concaténation** dans le corrigé distribué.

9.3 Correction minimale

Remplacer chaque `;` manquant par une concaténation verticale :

```
c = [c; a(i); b(i)];    % au lieu de [c a(i) b(i)]
c = [c; b(k+1:m)];      % au lieu de [c b(k+1:m)]
c = [c; a(k+1:n)];      % au lieu de [c a(k+1:n)]
```

```
version verticale -> [1 -2  3 -4  5  7]     % (serie1_pieges.m)
```

9.4 Version vectorisée (sans boucle de croissance)

Faire grandir `c` itération par itération est coûteux (MATLAB réalloue le tableau à chaque tour). On préalloue et on remplit par pas de 2 :

```
function c = interleave(a, b)
% Entrelace a et b. Tolere des tailles differentes et des vecteurs vides.
% Retourne toujours un vecteur COLONNE.
    a = a(:); b = b(:);              % on impose la forme colonne
    n = length(a); m = length(b);
    k = min(n, m);
    c = zeros(n + m, 1);             % preallocation
    c(1:2:2*k) = a(1:k);
    c(2:2:2*k) = b(1:k);
    if n > k
        c(2*k+1:end) = a(k+1:n);     % queue de a
    elseif m > k
        c(2*k+1:end) = b(k+1:m);     % queue de b
    end
end
```

L'appel et la sortie (`serie1_exercices.m`) :

```
a = [1; 3; 5; 7];
b = [-2; -4];
interleave(a, b)'
```

```
interleave([1 2 3], [10 20 30 40])'
interleave([], [1 2 3])'
```

```
ans = 1 -2 3 -4 5 7          % conforme a l'enonce (transpose d'un vecteur colonne)
ans = 1 10 2 20 3 30 40
ans = 1 2 3
```

À retenir.

- Réflexe `a = a(:); b = b(:)` : rendre la fonction insensible à l'orientation des entrées, puis imposer *une* forme de sortie (ici colonne).
- **Préallouer** (`zeros(n+m,1)`) avant toute boucle de remplissage ; `c(1:2:2*k) = a(1:k)` remplit les indices impairs, `c(2:2:2*k)` les indices pairs.
- Toujours tester les **cas limites** demandés par l'énoncé : vecteur vide (`min(0,m) = 0`), tailles inégales, les deux vides.

10. Tracer $y = x^2$ et $x = y^2$ dans la même fenêtre

Énoncé. Dessiner dans la même fenêtre les graphes de $y = x^2$ et de $x = y^2$ pour $-1 \leq x \leq 3$, avec légendes d'axes, titre, `grid` et `legend`.

10.1 Le point délicat : $x = y^2$ n'est pas une fonction

La courbe $x = y^2$ donne *deux* valeurs de y pour $x > 0$:

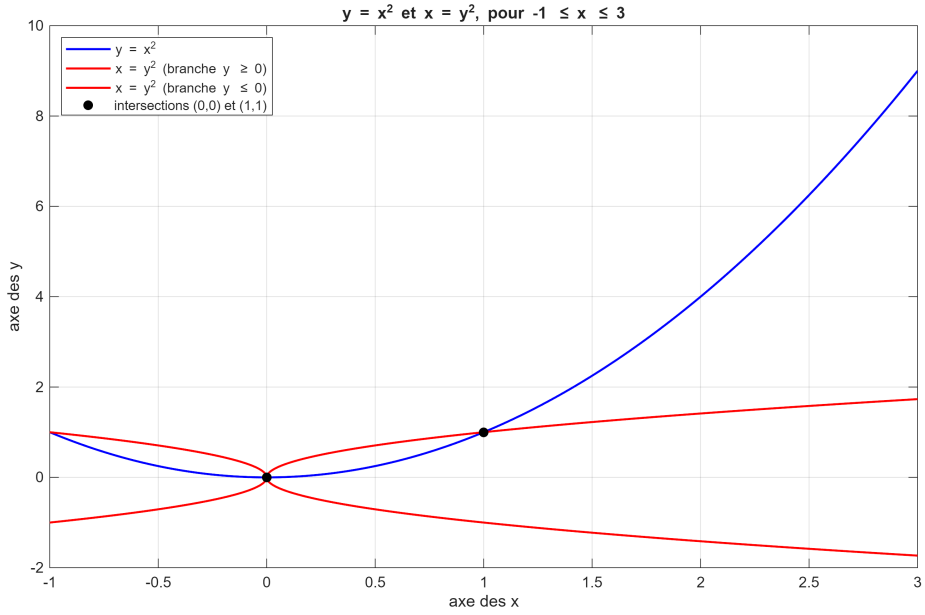
$$x = y^2 \iff y = \pm\sqrt{x}, \qquad \text{défini seulement pour } x \geq 0.$$

Il faut donc tracer deux branches. La solution du professeur utilise `sqrt(abs(x))`, ce qui permet de calculer quelque chose sur $[-1, 0]$ — mais la portion ainsi tracée n'appartient **pas** à la courbe. Vérification à l'exécution :

```
x = -1 : sqrt(abs(x)) = 1 alors que x = y^2 exige x >= 0
ecart max (sqrt(abs(x)))^2 - x sur [-1,0] : 2
```

10.2 Code retenu

```
x = -1 : 0.001 : 3;
y = sqrt(abs(x));
figure('Color', 'w')
plot(x, x.^2, 'b', 'LineWidth', 1.4); hold on
plot(x, y, 'r', 'LineWidth', 1.4)
plot(x, -y, 'r', 'LineWidth', 1.4)
plot([0 1], [0 1], 'ko', 'MarkerFaceColor', 'k', 'MarkerSize', 6)
xlabel('axe des x'); ylabel('axe des y')
title('y = x^2 et x = y^2, pour -1 \leq x \leq 3')
grid on
legend('y = x^2', 'x = y^2 (branche y \geq 0)', 'x = y^2 (branche y \leq 0)', ...
       'intersections (0,0) et (1,1)', 'Location', 'northwest')
```



10.3 Les deux points d'intersection

Résoudre $x^2 = x$ donne $x = 0$ ou $x = 1$: la parabole bleue coupe la branche $y = \sqrt{x}$ en $(0,0)$ et $(1,1)$. Sur $[-1,0]$, la branche supérieure tracée est horizontale à $y = 1$ ($\sqrt{|x|} = 1$ en $x = -1$) : c'est précisément l'artefact signalé ci-dessus.

À retenir.

- `'.-'` et non `'*'` : `y = x.^2` ; sans le point, MATLAB tente un produit matriciel de deux vecteurs ligne.
- Une courbe *non fonctionnelle* ($x = y^2$, cercle, …) se **paramètre** : `y = linspace(-sqrt(3), sqrt(3), 2000); x = y.^2;` donne la vraie courbe, sans bricolage de valeur absolue.
- Les quatre commandes d'habillage attendues : `xlabel`, `ylabel`, `title`, `grid`, auxquelles s'ajoute `legend` ; `hold on` pour superposer.
- Un marqueur ajouté après coup apparaît comme une entrée de légende (« data1 ») : lui donner son propre libellé, ou l'exclure de la légende.

11. Récapitulatif

Commande	Ce qu'il faut retenir
`format long` / `short`	affichage seulement ; 15 chiffres significatifs ; 17 pour un aller-retour exact
`eps`	$2^{-52} = 2{,}22 \times 10^{-16}$: granularité relative des doubles
`rat(x)` / `format rat`	fraction continue ; `[n,d] = rat(x, tol)` dépend de la tolérance ($10^{-6}\|x\|$ par défaut)
`x'` vs `x.'`	transposée conjuguée vs transposée simple
`x(:)`	aplatit en **colonne**, parcours colonne par colonne
`x*y'`	produit scalaire (1×1) ; `x'*y` = produit extérieur
`cumprod` / `prod`	produits cumulés / produit total ; exact jusqu'à 22!, `Inf` dès 171!
`magic(n)`	constante $(n^3 + n)/2$; $n \geq 3$ (magic(2) n'est pas magique)
`rem`, `all`, `any`	reste modulaire ; « pour tout » / « il existe »
`zeros` + indices pas de 2	préallouer plutôt que faire grandir `c = [c ...]`

Les cinq pièges qui coûtent des points

1. **Confondre affichage et valeur stockée** (`format long`, `format rat`) : le cours commence précisément par là, et le barème exige le *détail* du calcul.
2. **Comparer des puissances sans logarithmes** (e^{π} vs π^e).
3. **Se tromper de transposée** : `x*y` ne compile pas, `x'*y` n'est pas un scalaire, `.*` n'est pas `*`.
4. **Ne pas tester les cas limites** (vecteur vide, tailles inégales, $n = 2$ pour `magic`, entrée non entière pour `pair`) : c'est la moitié des exigences de l'énoncé 9.
5. **Faire grandir un vecteur dans une boucle** sans préallocation ; et tracer une courbe non fonctionnelle sans la paramétrer correctement (artefact de `sqrt(abs(x))`).

À connaître pour les travaux écrits

Deux TE de 60 minutes ; « une solution non développée sera considérée comme fausse » et le barème distingue méthode *et* valeur numérique (2 points = tout juste, 1 point = plusieurs erreurs d'importance). Chaque réponse doit donc montrer : la formule ou la commande employée, le calcul intermédiaire, puis le résultat arrondi correctement.